

这节课主要讲两件事：**Tokenization** 如何把原始文本切成模型能处理的 token；**Model Architecture** 尤其是 **Transformer** 如何把 token 变成上下文相关的表示，并用于理解或生成文本。

1. 为什么需要 Tokenization?

语言模型本质上不是直接处理“自然语言”，而是处理一串 token。也就是说，模型看到的不是：

| the mouse ate the cheese

而是类似：

| [the, mouse, ate, the, cheese]

所以，第一步就是把字符串切成 token，这个过程叫 **tokenization**。

通俗理解：

tokenizer 就像“切菜机”。原始文本是一整根黄瓜，模型不能直接吃，需要先切成片。怎么切，会影响模型吃得快不快、吸收得好不好。

2. 为什么不能简单按空格切词?

最简单的办法是：

```
text.split(" ")
```

但这有很多问题。

比如中文没有天然空格：

| 我今天去了商店。

如果按空格切，整句话可能就是一个 token，完全不合理。

再比如德语有很长的复合词，英语里也有 father-in-law、don't 这种词。

所以，**按空格切词太粗糙，无法适配多语言和复杂词形。**

一个好的 tokenization 要在两者之间平衡：

切法	问题
切得太细，比如按字符、字节切	序列太长，模型处理成本高
切得太粗，比如整词切	很多词无法共享信息，遇到新词也麻烦
比较理想	切成有统计意义或语言意义的子词单元

通俗理解：
不要把“西红柿炒鸡蛋”切成一个整体，也不要切成“西 / 红 / 柿 / 炒 / 鸡 / 蛋”这么碎，而是切成模型最容易学习的中间颗粒度。

3. BPE：最常见的子词切分方法之一

讲义重点介绍了 **Byte Pair Encoding, BPE** 。

它的直觉是：

一开始把文本切得很细，比如字符；然后不断把最常一起出现的相邻片段合并。

例如：

```
[t, h, e] → [th, e] → [the]
```

如果 “t+h” 经常一起出现，就合并成 “th”；
如果 “th+e” 经常一起出现，就合并成 “the”。

通俗理解：
BPE 像是在统计哪些字母或片段经常“搭伙过日子”，然后把它们合并成一个更大的单位。

BPE 的好处是，它能处理没见过的新词。
比如模型没见过 “unbelievable”，也可以拆成：

```
un + believe + able
```

这样模型仍然可以大致理解它。

4. Byte-level BPE：为什么有些 tokenizer 从字节开始？

讲义提到，Unicode 字符非常多，尤其是多语言场景下，训练数据不可能覆盖所有字符。

所以一种做法是：

不要从字符开始，而是从 byte，也就是字节开始做 BPE。

这也是 GPT 系列 tokenizer 的重要思路之一。

通俗理解：

字符是“语言层面的单位”，但字节是“计算机层面的单位”。

从字节开始，覆盖面更广，几乎任何文本都能表示，只是有时候切出来不太符合人的直觉。

5. SentencePiece / Unigram Model：另一种 tokenizer 思路

除了 BPE，讲义还介绍了 **SentencePiece** 里的 **Unigram Model**。

BPE 是一种比较贪心的合并方法：

“谁经常一起出现，我就合并谁。”

Unigram Model 更像是：

“我先准备一个比较大的候选词表，然后用概率模型判断哪些 token 更有用，逐步删掉不重要的 token。”

通俗理解：

BPE 像是从小积木不断拼成大积木。

Unigram Model 像是先准备一大堆积木，然后筛掉不好用的。

讲义里还提到，不同 tokenizer 会影响效率。例如 Jurassic 使用的 tokenizer 比 GPT-3 平均需要更少 token，因此同样上下文长度下能塞进更多文本。

这点很重要：

tokenizer 不只是预处理工具，它会直接影响模型速度、上下文容量和多语言表现。

6. 三类语言模型架构：Encoder-only、Decoder-only、Encoder-Decoder

讲义接着从 tokenization 进入模型架构，介绍了三类常见模型。

6.1 Encoder-only：代表是 BERT、RoBERTa

Encoder-only 模型主要用于“理解”文本。

它会给每个 token 生成一个上下文相关的 embedding，但不擅长直接生成长文本。

例子：

```
[CLS] the movie was great → positive
```

它适合做：

- 情感分类
- 文本分类
- 自然语言推理
- 句子匹配

它的优势是：

一个词表示可以同时看左边和右边。

比如句子：

| I saw a bat.

bat 是蝙蝠还是球棒，要看上下文。Encoder 可以同时看前后文，因此适理解任务。

通俗理解：

BERT 像一个阅读理解高手，它会把整篇文章读完，再判断每个词是什么意思。

6.2 Decoder-only：代表是 GPT-2、GPT-3

Decoder-only 模型就是我们熟悉的 GPT 类模型。

它的任务是：

| 给定前面的 token，预测下一个 token。

例如：

```
the movie was → great
```

它只能看左边，不能偷看右边。

因为生成文本时，未来的词还不存在。

优势是：

- 天然适合文本生成
- 训练目标简单：预测下一个 token

- 可以通过 prompt 完成很多任务

劣势是：

- 每个位置只能依赖左侧上下文
- 做纯理解任务时，有时不如 encoder-only 直接

通俗理解：

GPT 像一个接话高手。你说前半句，它根据前文不断往后续写。

6.3 Encoder-Decoder：代表是 T5、BART

Encoder-Decoder 同时包含理解端和生成端。

输入部分用 Encoder 双向理解；

输出部分用 Decoder 自回归生成。

适合：

- 翻译
- 摘要
- 表格转文本
- 问答生成

例如：

```
[name: Clowns | eatType: coffee shop] → Clowns is a coffee shop.
```

通俗理解：

Encoder-Decoder 像一个“先读懂材料，再写出答案”的系统。

7. 从 RNN 到 Transformer：为什么 Transformer 重要？

在 Transformer 之前，常见的序列模型是 RNN、LSTM、GRU。

RNN 的思路是从左到右读文本，每一步更新一个隐藏状态：

```
h1 → h2 → h3 → ...
```

问题是：
虽然理论上 RNN 可以记住很远的信息，但实际上长距离依赖很难学。
比如一句很长的话，开头的信息传到后面时可能已经“衰减”了。
通俗理解：
RNN 像一个人边走边记笔记，但走太远之后，前面的细节就模糊了。
LSTM 和 GRU 改善了这个问题，但没有彻底解决。
真正让大语言模型起飞的是 Transformer。

8. Transformer 的核心：Attention

Transformer 的核心机制是 **attention** 。

Attention 的本质可以理解为：

当前这个 token 想知道：我应该重点参考句子里的哪些 token？

例如句子：

The animal didn't cross the street because it was too tired.

这里 it 指的是 animal，不是 street。
Attention 可以让 it 去“关注” animal。
通俗理解：
Attention 就像阅读时拿荧光笔划重点。模型在处理某个词时，会自动判断哪些词更重要。

9. Attention 的三个角色：Query、Key、Value

讲义用“soft lookup table”解释 attention。
可以理解成：

角色	通俗解释
Query	我现在想找什么信息
Key	每个 token 提供的索引或标签
Value	每个 token 真正携带的信息

流程是：

1. 当前 token 生成一个 Query。
2. 所有 token 生成 Key 和 Value。
3. Query 和每个 Key 做匹配，得到相关性分数。
4. 分数经过 softmax 变成权重。
5. 用这些权重加权求和 Value，得到当前 token 的新表示。

通俗理解：

你问一个问题 Query，然后去资料库里匹配每条资料的 Key，最后把相关资料的 Value 按重要程度混合起来。

10. Self-Attention：每个 token 都和其他 token 交流

当每个 token 都拿自己当 Query，去关注整句话里的其他 token，这就叫 Self-Attention。

它的作用是：

让句子里的每个词都能和其他词“对话”。

例如：

```
[the, mouse, ate, the, cheese]
```

处理 mouse 时，它可能关注 the、ate；

处理 cheese 时，它可能关注 ate、mouse。

通俗理解：

Self-Attention 像开会，每个词都听听其他词在说什么，然后更新自己对当前句子的理解。

11. Multi-Head Attention：从多个角度看关系

一个 attention head 只能捕捉一种关系。

但语言里有很多关系：

- 语法关系
- 指代关系
- 主谓宾关系
- 语义相似关系
- 位置关系

所以 Transformer 会使用多个 attention head。

通俗理解：

Multi-head attention 就像多个专家同时读一句话。一个专家看语法，一个专家看指代，一个专家看语义，最后把他们的意见合并。

12. FeedForward 层：每个 token 独立加工

Self-Attention 负责 token 之间的信息交流。

FeedForward 层负责对每个 token 自己的表示进行加工。

讲义里的伪代码大致是：

```
yi = W2 max(W1 xi + b1, 0) + b2
```

不用纠结公式。

它本质上是一个小型神经网络，对每个位置单独做非线性变换。

通俗理解：

Attention 是开会交流，**FeedForward** 是每个人回到自己座位后整理笔记、深化理解。

13. Residual Connection 和 LayerNorm：让深层模型能训练

Transformer 可以堆很多层，比如 GPT-3 堆了 96 层。

但层数太深会导致训练困难，梯度可能消失或不稳定。

所以讲义介绍了两个关键技巧：

13.1 Residual Connection 残差连接

不是直接输出：

```
f(x)
```

而是输出：

```
x + f(x)
```

这样即使 $f(x)$ 学得不好，原始信息 x 也能传下去。

通俗理解：

残差连接像给信息开了一条高速旁路，避免它在很多层之后丢失。

13.2 Layer Normalization 层归一化

LayerNorm 的作用是让每一层的数值保持稳定，不要过大或过小。

通俗理解：

LayerNorm 像给每层输出做“体温检测和校准”，防止数值太激动或太低迷。

14. Transformer Block 的整体结构

一个 Transformer Block 大致包含：

```
Self-Attention
+ Add & Norm
+ FeedForward
+ Add & Norm
```

也就是：

1. 先让 token 之间交流。
2. 加残差连接和归一化。
3. 再让每个 token 独立加工。
4. 再加残差连接和归一化。

通俗理解：

一个 Transformer Block 就是一轮“集体讨论 + 个人消化”。堆很多层，就是反复讨论、反复深化。

15. Positional Embedding：为什么模型需要知道位置？

Self-Attention 本身不天然知道顺序。

如果没有位置信息，下面两句话可能看起来差不多：

```
the mouse ate the cheese
the cheese ate the mouse
```

但显然意思完全不同。

所以 Transformer 需要加入 **positional embedding** ，告诉模型每个 token 在序列中的位置。

通俗理解：

token embedding 告诉模型“这个词是什么”；**positional embedding** 告诉模型“这个词在哪里”。

16. GPT-3 架构：堆叠很多 Transformer Block

讲义最后用 GPT-3 举例。GPT-3 可以粗略理解为：

```
Token Embedding + Position Embedding
→ Transformer Block × 96
→ 预测下一个 token
```

讲义中列出的 GPT-3 关键配置包括：

参数	含义
hidden state dimension = 12288	每个 token 的内部表示维度
feed-forward dimension = 4 × hidden dimension	中间层更宽，用于增强表达能力
attention heads = 96	多个 attention 头
context length = 2048	最多处理 2048 个 token

通俗理解：

GPT-3 不是一个神秘黑箱，而是很多层 Transformer Block 堆起来的巨大“接话机器”。每一层都在重新理解上下文，最后预测下一个 token。

17. 这节课最重要的知识框架

可以把整节课压缩成下面这条链路：

```
原始文本
→ Tokenization
→ Token Embedding
→ Position Embedding
→ Transformer Blocks
```

→ Contextual Embeddings

→ 预测 / 分类 / 生成

大白话版：

一句话

→ 先切成模型能吃的小块

→ 每个小块变成向量

→ 加上位置信息

→ 经过很多轮“互相注意 + 独立加工”

→ 得到上下文理解

→ 用来生成文本或完成任务

18. 和大语言模型实践最相关的几个启发

启发一：Tokenizer 会影响模型能力和成本

同样一段文本，不同 tokenizer 切出来的 token 数可能不一样。

token 越多，模型处理越慢，上下文窗口消耗越快。

所以为什么中文、代码、数学、多语言场景经常会有 tokenizer 效率问题？

因为它们被切成 token 的方式可能不如英文自然。

启发二：GPT 本质上是 Decoder-only Transformer

ChatGPT、GPT-3、GPT-4 这类模型，本质上都是 Decoder-only 思路的延展：

根据前文预测后文。

只是随着模型规模、训练数据、后训练方法和工具调用能力增强，它们表现出了更复杂的能力。

启发三：Attention 是大模型理解上下文的关键

Attention 让模型不再像 RNN 那样只能一步步传递信息，而是可以让任意两个 token 直接建立联系。

这就是 Transformer 能处理长文本、复杂依赖关系的重要原因。

启发四：Transformer 的强大来自模块化堆叠

单个 Transformer Block 并不神秘。

核心就是：

```
Self-Attention + FeedForward + Residual + LayerNorm
```

真正的能力来自：

19. 最后用一个比喻总结

可以把大语言模型想象成一个大型读写系统：

1. **Tokenizer**：把文章切成便于处理的小纸片。
2. **Embedding**：给每张纸片编码成数字向量。
3. **Position Embedding**：标记每张纸片原来在文章里的位置。
4. **Attention**：每张纸片去看其他纸片，判断谁和自己关系最大。
5. **Multi-head Attention**：多个专家从不同角度判断关系。
6. **FeedForward**：每张纸片根据吸收的信息更新自己。
7. **Transformer Stack**：重复很多轮，理解越来越深。
8. **Decoder 输出**：根据前文预测下一个 token。

所以，这节课其实是在告诉你：

大语言模型不是直接“理解文字”，而是先把文字切成 token，再把 token 变成向量，通过 Transformer 反复计算上下文关系，最后根据概率生成下一个 token。